

Título del proyecto:
Simulador de sistema operativo

Alumnos:
Flores Alvarez Roberto Maximiliano
Gonzalez Morales Daniel Alejandro
Ramirez Olvera Erick Yael
Rosas Salazar Jose Eduardo

Facultad de Ingeniería. Universidad Autonoma de Mexico

Asignatura:
Estructura de datos y algoritmos 1

Grupo: 5

Profesora:
Adara Mercado Mendoza Ariel

Semestre:
2026-2

Fecha de entrega:
3 de junio del 2025

Índice

1. Introducción.....	2
2. Objetivos.....	3
3. Arquitectura del sistema.....	4
3.1 Estructura de Directorios.....	4
3.2 Flujo de Datos.....	4
3.3 Decisiones de Diseño.....	5
3.4 Modelo de procesos.....	5
4. Estructuras de Datos Implementadas.....	6
4.1 Pila (Stack).....	6
4.2 Cola FIFO (Queue).....	6
4.3 Cola Circular (CircularQueue).....	6
4.4 Lista Ligada Simple (LinkedList).....	7
4.5 Lista Doblemente Ligada(MemoryBlock).....	7
5. Planificadores de CPU (Schedulers.....	8
5.1 FIFO (First In First Out).....	8
5.2 Round Robin.....	8
5.3 SJF (Shortest Job First).....	8
6. Administración de Memoria.....	9
6.1 Algoritmos de Asignación.....	9
6.2 Coalescencia de Bloques Libres.....	9
6.3 Compactación de Memoria.....	10
7. Algoritmos Implementados.....	11
7.1 Fuerza Bruta.....	11
7.2 Greedy.....	11
7.3 Backtracking.....	11
7.4 Divide y vencerás - Mergesort.....	11
7.5 Programación Dinámica.....	12
8. Integración C/ Python.....	12
9. Conclusiones.....	12

1. Introducción

En este reporte se documenta el diseño, implementación y análisis de un simulador de sistema operativo desarrollado en lenguaje C con integración al lenguaje Python, como proyecto integrador del curso de Estructuras de Datos y Algoritmos I de la Universidad Nacional Autónoma de México, correspondiente al semestre 2026-2.

El desarrollo de este proyecto responde a la necesidad de aplicar de manera práctica los conceptos teóricos adquiridos a lo largo del curso. Un sistema operativo real administra recursos críticos como el procesador y la memoria mediante algoritmos y estructuras de datos bien definidas; este simulador réplica ese comportamiento a nivel conceptual, permitiendo observar y analizar cómo distintas estrategias de planificación y asignación impactan el rendimiento del sistema.

El sistema fue diseñado siguiendo principios de programación modular, ya que cada componente tiene una responsabilidad clara, una interfaz pública definida en archivos de cabecera (.h) y una implementación encapsulada en archivos de implementación (.c). Esta separación facilita el mantenimiento, las pruebas unitarias y la comprensión del código.

Los módulos implementados comprenden cinco áreas principales:

- Estructuras de datos: pila (Stack), cola FIFO (Queue), cola circular (CircularQueue), lista ligada simple (LinkedList) y lista doblemente ligada (MemoryBlock), cada una con su análisis de complejidad temporal y espacial.
- Planificadores de CPU: FIFO, Round Robin con quantum configurable y Shortest Job First (SJF), demostrando distintos criterios de decisión sobre qué proceso ejecutar en cada momento.
- Administración de memoria: algoritmos First Fit, Best Fit y Worst Fit para asignación, junto con coalescencia de bloques libres y compactación de memoria para combatir la fragmentación externa.
- Algoritmos: fuerza bruta, estrategias greedy, backtracking, divide y vencerás (Merge Sort) y programación dinámica (Mochila 0/1), cada uno con su recurrencia y análisis formal de complejidad.
- Integración C/Python: generación automatizada de procesos, benchmarking con medición de tiempos, validación matemática de resultados y generación de gráficas comparativas.

2. Objetivo General

Desarrollar un simulador de sistema operativo funcional que replique el comportamiento básico de un sistema operativo moderno, integrando los conocimientos adquiridos durante el curso de Estructuras de Datos y Algoritmos I.

El simulador deberá ser capaz de administrar procesos, memoria y planificación de ejecución mediante estructuras de datos y algoritmos correctamente implementados, analizados formalmente en términos de complejidad temporal y espacial, y validados mediante pruebas unitarias.

Objetivos específicos:

- Implementar las estructuras de datos lineales fundamentales, pila, cola, cola circular, lista ligada simple y lista doblemente ligada — demostrando su correcto funcionamiento y justificando su elección para cada módulo del sistema.
- Desarrollar tres planificadores de CPU (FIFO, Round Robin y SJF) que simulen distintos criterios de decisión sobre la asignación del procesador.
- Implementar un administrador de memoria con tres estrategias de asignación (First Fit, Best Fit y Worst Fit), coalescencia de bloques libres y compactación, capaz de detectar y reducir la fragmentación externa.
- Aplicar cuatro paradigmas algorítmicos — fuerza bruta, greedy, backtracking, divide y vencerás — y programación dinámica, documentando la recurrencia y complejidad de cada uno.
- Integrar C y Python en un flujo completo de generación, simulación, validación y análisis de resultados.
- Documentar el uso crítico de herramientas de inteligencia artificial durante el desarrollo, registrando prompts, respuestas, errores encontrados y correcciones realizadas.

3. Arquitectura del Sistema

El proyecto fue diseñado siguiendo una arquitectura modular organizada en capas, donde cada componente tiene una responsabilidad única y bien definida. Esta separación de responsabilidades facilita el mantenimiento, la reutilización de código y la realización de pruebas unitarias independientes por módulo.

La arquitectura se divide en dos niveles principales. El primer nivel son las interfaces públicas, definidas en archivos de cabecera (.h) dentro de la carpeta `include/`. Estos archivos declaran qué funciones y estructuras existen en el sistema sin revelar cómo están implementadas — aplicando el principio de encapsulamiento. El segundo nivel son las implementaciones, contenidas en archivos (.c) dentro de `src/`, donde reside la lógica real del sistema.

Esta separación tiene una ventaja concreta: cualquier módulo puede cambiar su implementación interna sin afectar al resto del sistema, siempre que respete la interfaz definida en su `.h`.

3.1 Estructura de Directorios

La organización del proyecto es la siguiente:

```
proyecto-final/
├── include/      ← Interfaces públicas (.h) – contratos del sistema
├── src/
│   ├── memory/  ← Administrador de memoria (First Fit, coalescencia)
│   ├── scheduler/ ← Planificadores de CPU (FIFO, Round Robin, SJF)
│   ├── algorithms/ ← Algoritmos (greedy, backtracking, DP, merge sort)
│   └── *.c       ← Estructuras de datos (stack, queue, linked list)
├── tests/        ← Pruebas unitarias por módulo
├── scripts/      ← Benchmarking y análisis en Python
├── reports/
│   ├── csv/      ← Resultados numéricos del benchmarking
│   └── png/       ← Gráficas generadas
├── docs/         ← Bitácora de IA y documentación
├── bin/          ← Ejecutables generados por el compilador
└── Makefile      ← Automatización de compilación
```

3.2 Flujo de Datos

El sistema opera en un flujo bien definido entre C y Python:

1. Python genera procesos aleatorios con `generate_process.py` y los guarda en CSV.
2. C recibe los parámetros, ejecuta la simulación completa y produce resultados.
3. Python mide los tiempos de ejecución con `benchmark.py` usando `subprocess`.
4. Python valida la corrección matemática de los resultados con `validate_results.py`.
5. Python genera gráficas comparativas con `graphs.py` usando `matplotlib`.

3.3 Decisiones de Diseño

Durante el desarrollo se tomaron decisiones de diseño importantes:

- Se usó lista doblemente ligada para la memoria porque la coalescencia necesita acceder al bloque anterior en $O(1)$ — imposible con lista simple.
- Se usó cola circular con arreglo para Round Robin en lugar de lista enlazada, porque el acceso por índice con módulo es más eficiente para reinsertar elementos.
- Se usó tipo opaco (`typedef struct Stack Stack`) en las estructuras de datos para que los detalles de implementación no sean visibles desde otros módulos.

3.4 Modelo de proceso

Cada proceso se representa con el siguiente struct, que contiene su identificador único, tiempos de ejecución, prioridad, requerimiento de memoria y estado actual:

Campo	Tipo	Descripción
pid	int	Identificador unico del proceso
burst_time	int	Tiempo total de CPU requerido
remaining_time	int	Tiempo restante de ejecucion
priority	int	Prioridad del proceso (1-5)
memory_required	int	Memoria solicitada en MB
state	ProcessState	Estado actual: READY/RUNNING/BLOCKED/FINISHED

4. Estructuras de datos implementadas

4.1 Pila (Stack)

Esta parte fue implementada con lista enlazada en src/stack.c. Cada nodo contiene un valor entero (PID) y un puntero al siguiente nodo. La estructura interna es opaca y sólo se accede a través de las funciones definidas en el header. El uso en el sistema fue para el historial de ejecución de procesos. Cada vez que un proceso se ejecuta, su PID se registra con `stack_push()`. Permite reconstruir el orden de ejecución en cualquier momento.

Operación	Complejidad temporal	Complejidad espacial
<code>stack_push()</code>	$O(1)$	$O(1)$
<code>stack_pop()</code>	$O(1)$	$O(1)$
<code>stack_peek()</code>	$O(1)$	$O(1)$
<code>stack_destroy()</code>	$O(n)$	$O(1)$

4.2 Cola FIFO (Queue)

Esta parte fue implementada con lista enlazada con dos punteros: front (salida) y rear (entrada), lo que garantiza un orden FIFO, ya que el primer elemento en entrar es el primero en salir. Su uso en el sistema fue para `scheduler FIFO`. Los procesos se encolan en orden de llegada y se ejecutan en ese mismo orden sin interrupción.

Operación	Complejidad temporal	Complejidad espacial
<code>enqueue()</code>	$O(1)$	$O(1)$
<code>dequeue()</code>	$O(1)$	$O(1)$
<code>queue_peek()</code>	$O(1)$	$O(1)$

4.3 Cola Circular (CircularQueue)

Esta parte fue implementada con un arreglo dinámico y aritmética de módulo. Los índices head y tail avanzan con la fórmula $(index + 1) \% capacidad$, produciendo el efecto circular. Cuando se llena, se redimensiona duplicando la capacidad. Su uso en el sistema fue para `Scheduler Round Robin`. Permite reinsertar procesos al final de forma eficiente cuando no terminan en su quantum.

Operación	Complejidad temporal	Complejidad espacial
<code>cq_enqueue()</code>	$O(1)$	$O(1)$
<code>cq_dequeue()</code>	$O(1)$	$O(1)$
<code>cq_resize()</code>	$O(n)$	$O(n)$

4.4 Lista Ligada Simple (LinkedList)

La lista ligada con inserción ordenada por PID. Cada nodo almacena el PID y el burst_time del proceso terminado. La función ll_insert_sorted() recorre la lista hasta encontrar la posición correcta para mantener el orden. El uso en el sistema fue almacenar procesos terminados ordenados por PID para facilitar consultas y reportes.

Operación	Complejidad Temporal	Complejidad Espacial
ll_insert_sorted()	$O(n)$	$O(1)$
ll_find()	$O(n)$	$O(1)$
ll_remove()	$O(n)$	$O(1)$

4.5 Lista Doblemente Ligada (MemoryBlock)

Cada bloque de memoria es un nodo con punteros next y prev. El puntero prev es fundamental para la coalescencia, ya que permite acceder al bloque anterior en $O(1)$ para verificar si también está libre y fusionarlos.

Operación	Complejidad Temporal	Complejidad Espacial
Inserción/Division	$O(1)$	$O(1)$
Búsqueda de bloque	$O(n)$	$O(1)$
Coalescencia	$O(n)$	$O(1)$
Compactación	$O(n)$	$O(1)$

5. Planificaciones de CPU (Schedulers)

5.1 FIFO (First in First Out)

El proceso que llega primero es el primero en ejecutarse. No hay interrupciones, ya que cada proceso corre hasta completar su burst_time. Es el algoritmo más simple pero puede causar el convoy effect, donde procesos largos retrasan a los cortos.

Complejidad: $O(n)$ temporal, $O(n)$ espacial.

5.2 Round Robin

Cada proceso recibe un quantum de tiempo de CPU. Si no termina en ese tiempo, se reinserta al final de la cola circular y espera su siguiente turno. Es el algoritmo más justo para sistemas de tiempo compartido.

Ejemplo con quantum=2 y procesos P1(5), P2(3), P3(4):

Turno	Proceso	t inicial	t final	Ejecutado	Restante
1	P1	0	2	2	3
2	P2	2	4	2	1
3	P3	4	6	2	2
4	P1	6	8	2	1
5	P2	8	9	1	FINISHED
6	P3	9	11	2	FINISHED
7	P1	11	12	1	FINISHED

5.3 SJF (Shortest Job First)

Ordena los procesos por burst_time ascendente antes de ejecutar, usando Merge Sort. El proceso más corto va primero. Es óptimo en tiempo de espera promedio entre los algoritmos no apropiados.

Complejidad de ordenamiento: $T(n) = 2T(n/2) + O(n) \Rightarrow O(n \log n)$ por Teorema Maestro caso 2.

6. Administración de Memoria

La memoria se representa como una lista doblemente ligada de bloques. Al inicializar, existe un solo bloque libre de tamaño `total_memory`. Conforme se asignan y liberan procesos, la lista crece y se fragmenta.

6.1 Algoritmos de Asignación

Algoritmo	Criterio de Selección	Ventaja	Desventajas
First Fit	Primer bloque que alcance	Rápido $O(n)$	Puede fragmentar el inicio
Best Fit	Bloque más pequeño que alcance	Minimiza fragmentación interna	Crea muchos bloques pequeños
Worst Fit	Bloque más grande disponible	Maximiza espacio restante	Puede desperdiciar bloques grandes

6.2 Coalescencia de Bloques Libres

La coalescencia es el proceso de fusionar bloques de memoria libre adyacentes en uno solo, reduciendo la fragmentación externa sin mover datos. Se ejecuta automáticamente cada vez que se llama a `mm_free()`, garantizando que la memoria se mantenga consolidada tras cada liberación.

El algoritmo recorre la lista doblemente ligada de izquierda a derecha. Cuando encuentra dos bloques libres consecutivos, absorbe el segundo dentro del primero sumando sus tamaños, actualizando el puntero `next` y liberando el nodo absorbido. El puntero `prev` de la lista doblemente ligada es fundamental aquí: permite verificar y actualizar el bloque anterior en $O(1)$, algo imposible con una lista simple.

```
Antes:  [P1:200] → [Libre:100] → [Libre:200] → [P3:100]
Paso 1: fusionar Libre:100 + Libre:200 → Libre:300
Después: [P1:200] → [Libre:300] → [P3:100]
```

El algoritmo no avanza al siguiente nodo cuando funciona, ya que este se queda en el mismo por si hay más bloques libres consecutivos. Solo avanza cuando el bloque actual o el siguiente están ocupados.

6.3 Compactación de Memoria

La compactación es una operación más agresiva que la coalescencia — en lugar de solo fusionar bloques libres, reorganiza físicamente todos los bloques ocupados moviéndose al inicio de la memoria, dejando un único bloque libre al final. Esto elimina completamente la fragmentación externa.

`mm_compact()` opera en dos fases. En la primera recorre la lista y asigna las direcciones `start` de todos los bloques ocupados de forma contigua desde el inicio. En la segunda elimina todos los bloques libres intercalados y crea un único bloque libre al final con toda la memoria disponible restante.

```
Antes:  [P1:200] → [Libre:300] → [P3:100] → [Libre:424]
        Memoria libre total: 724 MB, pero fragmentada en 2 bloques

Después: [P1:200] → [P3:100] → [Libre:724]
        Memoria libre total: 724 MB, en un único bloque continuo
```

La diferencia con la coalescencia es que la compactación puede crear espacio contiguo incluso cuando los bloques libres no son adyacentes, pero tiene un costo mayor porque reorganiza toda la lista.

Análisis de complejidad:

```
T(n) = O(n)   - dos recorridos lineales de la lista
S(n) = O(1)   - sin estructuras auxiliares
```

Operación	Cuando usarla	Costo
Coalescencia	Después de cada <code>mm_free()</code>	$O(n)$ — automática
Compactación	Cuando falla una asignación por fragmentación	$O(n)$ — bajo demanda

7. Algoritmos Implementados

7.1 Fuerza Bruta

`brute_force_find_block()` recorre todos los bloques de memoria uno por uno hasta encontrar uno libre suficientemente grande. Es el enfoque más simple pero revisa todas las posibilidades sin atajos.

Complejidad: $T(n) = O(n)$, $S(n) = O(1)$

7.2 Greedy

Los tres algoritmos de asignación de memoria (First Fit, Best Fit, Worst Fit) son estrategias greedy — toman una decisión local óptima en cada paso sin reconsiderar. `greedy_sjf_sort()` también es greedy al priorizar siempre el proceso más corto.

Complejidad: $T(n) = O(n)$ para asignación, $O(n \log n)$ para SJF sort.

7.3 Backtracking

`backtrack_allocate()` intenta asignar memoria a cada proceso en orden. Si un proceso no cabe, lo marca como BLOCKED y continua. Si asigna pero luego falla, deshace la asignación y retrocede. Esto garantiza explorar todas las combinaciones posibles.

Complejidad: $T(n) = O(2^n)$ peor caso, $S(n) = O(n)$ por la pila de llamadas recursivas.

7.4 Divide y Venceras — Merge Sort

`merge_sort_processes()` divide recursivamente el arreglo de procesos a la mitad hasta tener subarreglos de un elemento, luego los combina en orden ascendente de `burst_time`.

Recurrencia: $T(n) = 2T(n/2) + O(n)$

Solución por Teorema Maestro (caso 2, $a=2$, $b=2$, $f(n)=n$): $T(n) = O(n \log n)$

7.5 Programación Dinámica — Mochila 0/1

Resuelve el problema de maximizar cuantos procesos caben en memoria sin exceder la capacidad total. Cada proceso o se incluye (1) o no se incluye (0) — no se puede partir.

Bottom-Up: llena una tabla `dp[i][cap]` de forma iterativa, de menor a mayor.

Top-Down: usa recursion con memoización — guarda resultados en `memo[][]` para no recalcular.

Complejidad: $T(n) = O(n*W)$, $S(n) = O(n*W)$ donde W = capacidad de memoria.

8. Integración C/ Python

El sistema integra C y Python siguiendo el flujo establecido en la especificación del proyecto:

Script	Funcion	Librería Principal
generate_process.py	Genera procesos aleatorios y los guarda en CSV	random, csv
benchmark.py	Ejecuta el simulador C y mide tiempos con subprocess	subprocess, time, pandas
validate_results.py	Verifica corrección matemática con assert	Ninguna (Python puro)
graphs.py	Lee el CSV y genera gráficas PNG	matplotlib, numpy, pandas

9. Conclusiones

Roberto Maximiliano Flores Alvarez

La realización de este proyecto me permitió comprender de forma más profunda la importancia de las estructuras de datos en el desarrollo de sistemas complejos. Pude observar cómo cada estructura cumple una función específica dentro del simulador y cómo una elección adecuada impacta directamente en la eficiencia del sistema. Además, fortalecí mis habilidades de programación en C y trabajo colaborativo.

Daniel Alejandro Gonzalez Morales

Durante el desarrollo del simulador adquirí una mejor comprensión de los algoritmos de planificación y administración de memoria utilizados en los sistemas operativos. Analizar su complejidad temporal y espacial me ayudó a entender la relación entre teoría y práctica, así como la importancia de diseñar soluciones eficientes para la gestión de recursos.

Erick Yael Ramirez Olvera

Este proyecto me permitió reforzar conocimientos sobre programación modular, abstracción y análisis de algoritmos. La implementación de estructuras dinámicas y la integración entre C y Python representaron un reto que contribuyó a mejorar mis habilidades de desarrollo y depuración de software. Asimismo, comprendí mejor la forma en que un sistema operativo administra procesos y memoria de manera eficiente.

Jose Eduardo Rosas Salazar

La elaboración de este proyecto me ayudó a aplicar de manera práctica los conceptos vistos durante el curso y a desarrollar habilidades para resolver problemas mediante distintos paradigmas algorítmicos. También comprendí la importancia de la documentación, las pruebas y el trabajo en equipo para construir proyectos de software funcionales y bien estructurados.